lab0 实验指导书

环境、工具链与「从源码到跑起来」

0. 先读这一页：这份指导书怎么用

0.1 这门课要教你的判断力

你可能会想：**"环境都装好了，我为什么不直接 `make run` 就完事？"**

因为 lab0 的目标**不是"把程序跑起来"**，而是让你建立一套贯穿整个课程的判断力：

你以后一定会遇到的现象	需要的能力	lab0 教不教
`make build` 报一堆 `undefined reference`	分清是**编译期**还是**链接期**问题	✅ 教
`make run` 黑屏，什么都不打印	知道去查**串口/控制台**这条链路	✅ 教
想把内核暂停在 `main()` 第一行看看栈什么样	会用 **gdb + 远程调试**	✅ 教
看到 `missing QEMU dependency` 之类的怪错	知道**工具链和 QEMU 是两套东西**	✅ 教

0.2 本实验的四条主线

```
┌──────────────────────────────────────────────────────────────┐
│  主线 1   三层世界观          源码层 / 构建层 / 运行层，分清谁是谁          │
│  主线 2   交叉编译            为什么在你的电脑上要装"别的架构"的编译器      │
│  主线 3   Makefile            一条 make build 到底做了什么，逐段拆开        │
│  主线 4   gdb 调试            让内核停下来给你看，原理 + 操作 + 实测        │
└──────────────────────────────────────────────────────────────┘
```

0.3 阅读约定

标记	含义
📖	需要你**读代码**并停下来想一下的地方
⌨️	需要你**亲手敲命令**的地方
🧠	一个**隐藏知识点**——指导书默认你没学过，会补一句人话解释
⚠️	**易错点**，不看会踩坑
✍️	**动手任务**，做完才算过

> 📏 **关于行号**：全文引用的行号都是**骨架版**行号。你逐个任务填入代码后，后面的行号会整体下移——始终以你手上的文件为准。
>
> | 🎯 | **检验点**，答不上来就回头重读 |

0.4 时间预算

环节	建议时长
第 1 章 验证环境（把 xv6 跑起来）	15 min
第 2 章 三层世界观	15 min
第 3 章 交叉编译	20 min
第 4 章 Makefile 精读 + 动手 A	35 min
第 5 章 链接脚本与内核落点 + 动手 B	25 min
第 6 章 gdb 调试 + 动手 C	35 min
第 7 章 全链路总图 + 思考题	15 min

合计约 **2.5 小时**

---

1. 你现在拥有什么：预装虚拟机速览

1.1 开箱即用的环境清单

课程虚拟机里**已经装好了**下面这些东西。**你不需要重新搭环境，一个命令都不用装。**

组件	在系统里的位置/命令	它是什么（🧠 人话）
交叉编译器	`riscv64-unknown-elf-gcc`	能生成 **RISC-V 指令**的 C 编译器
交叉汇编器	`riscv64-unknown-elf-as`	把 `.S` 汇编文件翻成机器码
交叉链接器	`riscv64-unknown-elf-ld`	把一堆 `.o` 拼成一个完整的 `kernel`
二进制工具	`riscv64-unknown-elf-objcopy` / `objdump`	格式转换 / 反汇编，看机器码的"显微镜"
交叉调试器	`gdb-multiarch`	能调试 RISC-V 程序的 gdb
模拟器	`qemu-system-riscv64`	用软件**假装**自己是一台 RISC-V 电脑
构建工具	`make`	按规则决定"哪些文件需要重新编译"
外设工具	`perl`、`mkfs.vfat`、`dd`	生成 syscall 桩代码 / 造 FAT32 镜像 / 造空文件
固件	QEMU 自带 `-bios default`（OpenSBI）	开机第一段代码，帮你把机器初始化好

⚠️ **注意最后一个**：lab0 用的是 **QEMU 自带的 OpenSBI**，不是仓库里 `bootloader/` 目录的 RustSBI。所以我们**不需要**编译任何 bootloader。

1.2 三步验证：把 xv6 跑起来

⌨️ **第一步：确认你的工具链认识自己**

```
riscv64-unknown-elf-gcc --version     # 期望输出 riscv64-unknown-elf-gcc (GCC) 版本号
qemu-system-riscv64 --version         # 期望输出 QEMU emulator version 7.0 或更高
make --version                        # 期望输出 GNU Make 4.x
perl -v                               # 期望输出 This is perl 5...
```

🎯 **检验点 1**：如果 `riscv64-unknown-elf-gcc` 报 `command not found`，说明什么？

> 顺序想：**编译器没装**？还是**装了但不在 `PATH` 里**？还是**装了但叫另一个名字**？这三种情况怎么区分？（提示：`which`、`ls /usr/bin | grep riscv`、`echo $PATH`）

⌨️ **第二步：构建内核**

```
cd /home/ubuntu/Desktop/xv6-ai-labs-v3.1/lab0        # 虚拟机里实验代码的位置，以课程实际路径为准
make build
```

`make build` 干了三件事（第 4 章会拆开看）：

1. 编译并链接出内核 `target/kernel`
2. 生成反汇编 `target/kernel.asm` 和符号表 `target/kernel.sym`
3. 编译全部用户程序（`xv6-user/` 下的 `sh`、`ls`、`cat`…）

成功的标志：命令结束、回到 shell 提示符、**没有任何 `Error` / `error:`**。

⚠️ **`make build` 输出里出现 `warning` 可以忽略吗？**
分两种：**编译器（gcc）的 warning** 会被 `CFLAGS` 里的 `-Werror` 当成错误，build 直接失败（`infinite-recursion` 被 `-Wno-error` 豁免是唯一例外）；**链接器（ld）的 warning** 不归 `-Werror` 管——所以正常构建时你也会看到十几条 `LOAD segment with RWX permissions` 警告，这是链接脚本/工具链的既有现象，**不是你的错，可以忽略**。

⌨️ **第三步：生成文件系统镜像，然后运行**

```
make fs       # 生成 FAT32 镜像 fs.img（需要 sudo，可能要输密码，默认密码为123456）
make run      # 启动 QEMU，跑内核
```

⚠️ **sudo 只发生在 `make fs` 内部**（挂载/拷贝那几步会要密码）。**不要**把整条命令包成 `sudo make fs`——那样生成的 `fs.img` 会变成 root 所有，接下来 `make run` 会报 `Could not open 'fs.img': Permission denied`。

成功的标志：屏幕上先出现一个 ASCII 艺术字的 logo，然后是一串内核初始化打印，最后出现：

```
-> / $
```

这个 `-> / $` 就是 **xv6 shell 的提示符**。能敲命令了：

```
ls          # 列目录
echo hello  # 回显
```

🎯 **检验点 2**：`make run` 依赖 `make fs` 吗？打开 `Makefile` 找 `run:` 那一行看看它后面跟了什么（第 4.4 节会讲）。

1.3 怎么退出 QEMU

⚠️ **这是最常被问到的问题。** QEMU 用 `-nographic` 模式跑，它**抢走了你的终端**，键盘输入全部送给虚拟机。所以：

> **先按 `Ctrl` + `A`，松开，再按 `X`**

（不是 `Ctrl+A` 然后 `Ctrl+X`，是**依次**：`Ctrl+A`，然后单独按 `X`）

🧠 **为什么这么别扭？** 因为 `-nographic` 模式下 QEMU 要独占键盘给**虚拟机里的 xv6** 用——你在 xv6 shell 里敲的每个字符，都得原样送给内核。那怎么留给"宿主机 QEMU"一个逃生通道？QEMU 用了一个**转义前缀键**：`Ctrl+A`。先按它，下一个键就解释成"QEMU 自己的命令"，而不是给虚拟机的输入。`X` 就是 QEMU 的"退出"命令。

> 这个概念在计算机网络里会再遇到一次：**带内信令（in-band signaling）**与**转义（escaping）**。SSH 的 `~.` 退出、PPP 的 `0x7D` 转义、HTTP 的 `\r\n\r\n`，都是同一个家族。

---

2. 三层世界观：源码层 / 构建层 / 运行层

2.1 为什么必须先建立这个模型

零基础同学最典型的困惑是：**"我改的文件"和"我看到的运行结果"之间，到底隔了几步？**

答案是 **三层**：

```
┌──────────────────────────────────────────────────────────────────┐
│  ① 源码层（Source）                                              │
│     你在编辑器里看到的 .c / .S / .h / Makefile / .ld             │
│     特征：人写的、可读、改了就"变了"                             │
├──────────────────────────────────────────────────────────────────┤
│  ② 构建层（Build）                                               │
│     .o 目标文件 → 链接 → target/kernel（ELF 可执行文件）         │
│     用户程序：halt.c → halt.o → _halt → 打包进 fs.img            │
│     特征：机器码、不可读、只在 make 时更新                       │
├──────────────────────────────────────────────────────────────────┤
│  ③ 运行层（Run）                                                 │
│     QEMU 把 target/kernel 装进（虚拟的）物理内存，CPU 开始取指执行 │
│     特征：寄存器在变、内存在变、屏幕在刷                         │
└──────────────────────────────────────────────────────────────────┘
```

三层的关键规则：

1. **只有第 ② 层会"过期"**。你改了源码，第 ② 层的产物**不会自动更新**，必须重新 `make`。
2. **第 ③ 层每次运行都是"重新开始"**。QEMU 一退出，内存状态全丢；`fs.img` 是磁盘，会**保留**。
3. **排错要判断"问题在哪一层"**。这是最有价值的能力：

现象	问题层	想什么
`error: expected ';' before '}' token`	① 源码层	C 语法错了
`undefined reference to 'sys_halt'`	② 构建层	函数声明了但没实现/没链接进去
编译过了，但运行起来行为不对	③ 运行层	逻辑错误，要用 gdb 看
`riscv64-unknown-elf-gcc: command not found`	环境	工具链问题，不是代码问题
`make: *** No rule to make target ...`	② 构建层（Makefile 层）	Makefile 规则写错了

🎯 **检验点 3**：下面三条报错，分别属于哪一层？

```
(a) error: 'uint64' undeclared (first use in this function)
(b) qemu-system-riscv64: -drive file=fs.img: Could not open 'fs.img'
(c) target/kernel: No such file or directory
```

2.2 一个例子吃透三层：`halt` 这个命令是怎么来的

lab1 会让你实现一个 `halt` 命令（功能：关机）。我们先在这里**只追它的链路**，当成三层的教学样本。

```
第 ③ 层 运行层
   你在 xv6 shell 里敲 halt + 回车
        ↓    （shell 在 fs.img 里找到文件 /bin/halt，加载执行）
   执行到 halt 里的 ecall 指令……

第 ② 层 构建层
   xv6-user/halt.c  ──gcc──▶  halt.o   ┐
   xv6-user/ulib.o                     ├──ld──▶ _halt   ──copy──▶  fs.img 里的 /bin/halt
   xv6-user/usys.o（含 halt 桩）        ┘
   xv6-user/printf.o

第 ① 层 源码层
   xv6-user/halt.c       主程序：调 halt()，然后打印 "should not reach here"
   xv6-user/usys.pl      脚本，生成 halt 的用户态桩（li a7, SYS_halt / ecall / ret）
   kernel/include/sysnum.h   定义 SYS_halt 的编号
   kernel/syscall.c      分发表：编号 → 内核处理函数
   kernel/sysproc.c      内核处理函数 sys_halt() 的实现
```

三层追踪的意义：以后你改了 `xv6-user/` 下任何一个文件，都必须**重新打包 `fs.img`**，否则第 ③ 层看到的还是旧的 `/bin/halt`。这就是为什么 `Makefile` 里 `$U/_%` 和 `fs:` 是**两条不同的规则**。

⚠️ **最高频的新手坑**：改了 `xv6-user/halt.c`，只跑了 `make build`，然后 `make run`，发现行为没变。

> 原因：`make build` 只更新了内核和用户程序**在源码目录里的产物**，但**没有把它们重新拷进** `fs.img`。`fs.img` 是磁盘镜像，是第 ③ 层真正读的东西。
> 正确做法：先 `make fs`，再 `make run`（注意：`run` 目标只依赖 `build`，**不会**自动帮你刷新 `fs.img`）。第 4.4 节会详细拆。

🎯 **检验点 4**：把 `halt.c → _halt → fs.img 里的 /bin/halt` 这条链路背下来，用一句话说清"我改了 `halt.c`，为什么 `make build` 完还得重新打包文件系统"。

---

3. 交叉编译环境：为什么你的电脑上装了一个"别的架构"的编译器

3.1 一个反直觉的现象

你在一台 **x86-64 的笔记本**（或 Intel 虚拟机）上，装了一个名字里带 `riscv64` 的编译器。

这意味着：**这个编译器"说"的语言，和你电脑 CPU"听"的语言，不是同一种。**

名词	说的是什么
宿主平台（host）	正在**运行编译器**的机器：x86-64 + Linux
目标平台（target）	编译器**产出代码**要跑的机器：riscv64 + 无操作系统
交叉编译（cross-compile）	host ≠ target 的编译

🧠 **为什么不能直接用系统的 `gcc`？**
你系统里的 `gcc` 是 **native 编译器**，它的 target 就是 host（x86-64）。用它编出的机器码是 `mov %rax, %rbx` 这种 x86 指令，而 QEMU 模拟的 RISC-V CPU **根本不认识**。指令集不同，就像你不能拿一本英文书让只懂中文的人读。

🧠 **那为什么不用本机的 gcc + 自己写个后端？**
理论上可以，但工作量等于自己写一个编译器。交叉工具链就是**别人已经帮你写好的**：它内部知道"RISC-V 有哪些指令、寄存器叫什么、ABI 怎么传参"。

3.2 本实验的交叉工具链"五件套"

打开 `Makefile` 第 68~74 行：

```
TOOLPREFIX  := riscv64-unknown-elf-
# TOOLPREFIX    := riscv64-linux-gnu-
CC = $(TOOLPREFIX)gcc
AS = $(TOOLPREFIX)gas
LD = $(TOOLPREFIX)ld
OBJCOPY = $(TOOLPREFIX)objcopy
OBJDUMP = $(TOOLPREFIX)objdump
```

🧠 **`TOOLPREFIX` 是什么？** 它只是个**字符串前缀**。Makefile 里 `$(TOOLPREFIX)gcc` 展开后就是 `riscv64-unknown-elf-gcc`。这样设计的好处：换一套工具链，只改**一行** `TOOLPREFIX`，不需要改 20 处命令名。这就是**"配置与逻辑分离"**——你在软件工程里会反复见到这个思想。

🧠 **`unknown-elf` 是什么意思？** 交叉工具链的名字遵循 `arch-vendor-os-abi` 格式：

```
riscv64  -  unknown  -  elf
   ↑          ↑         ↑
 目标架构    厂商(未知)   目标操作系统 = 无操作系统(裸机 ELF)
```

对照着看另一个被注释掉的：`riscv64-linux-gnu-` = 目标是 **Linux 系统**下的 RISC-V 程序。
为什么本课程不能用它？ 因为 xv6 是**裸机内核**——它自己就是操作系统，没有 Linux 在下面撑着。用 `linux-gnu` 工具链编出来的东西会**默认链接 glibc**、默认假设有 Linux 系统调用可用，而裸机上这些都不存在。

🎯 **检验点 5**：下面四件事，分别用五件套里的哪个工具做？

任务	工具
把 `main.c` 编成 `main.o`	?
把 `main.o` 和一堆 `.o` 拼成 `target/kernel`	?
把 `target/kernel` 反汇编成人类可读的汇编，存到 `kernel.asm`	?
从 `initcode.out` 里剥掉符号信息，只留纯二进制 `initcode`	?

（答案：`gcc` / `ld` / `objdump` / `objcopy`）

3.3 亲手验证：看一眼"编译到目标平台"到底意味着什么

⌨️ 验证 1：看编译器自报家门

```
riscv64-unknown-elf-gcc -dumpmachine
# 期望输出: riscv64-unknown-elf  ← 这就是它默认的 target

gcc -dumpmachine
# 期望输出: x86_64-linux-gnu（或类似）← 你本机的 native target
```

两个输出不一样，这就是"交叉"的全部含义。

⌨️ 验证 2：亲自编一个最小程序，看它产出的机器码

```
cd /tmp
cat > hello.c <<'EOF'
int add3(int a, int b, int c) {
    return a + b + c;
}
EOF

# 用交叉编译器编 RISC-V 版
riscv64-unknown-elf-gcc -O1 -c hello.c -o hello_rv.o

# 反汇编看它长什么样
riscv64-unknown-elf-objdump -d hello_rv.o
```

你会看到类似这样的输出（这才是 RISC-V 的样子）：

```
0000000000000000 <add3>:
   0:   00b50533                add     a0,a0,a1
   4:   00c50533                add     a0,a0,a2
   8:   8082                    ret
```

再来编个 x86 版对照：

```
gcc -O1 -c hello.c -o hello_x86.o
objdump -d hello_x86.o
```

```
0000000000000000 <add3>:
   0:   8d 04 37                lea    (%rdi,%rsi,1),%eax
   3:   01 d0                   add    %edx,%eax
   5:   c3                      ret
```

🎯 **检验点 6**：这两段反汇编有三处明显不同，找出来。

> 提示：①指令助记符（`add` / `lea` vs `add`）②寄存器名字（`a0` vs `%eax`）③**指令长度**——RISC-V 是定长还是变长？x86 呢？
> 引申：`RISC` = Reduced Instruction Set Computer（精简指令集），`CISC` = Complex Instruction Set Computer（复杂指令集）。RISC-V 名字里的 **RISC** 就是这个意思。

3.4 还有一个"隐藏的 target"：`-mcmodel=medany`

打开 `Makefile` 第 78 行：

```
CFLAGS += -mcmodel=medany
```

🧠 **这是干什么的？** 它告诉编译器：**"代码和数据可能被放在 2GB 范围之外，请用能寻址到远方的寻址方式。"**

为什么需要？ 因为本实验内核被链接到 `0x80200000`（约 2.05 GB 处，见第 5 章），超过了 32 位有符号偏移能覆盖的范围。默认的 `-mcmodel=medlow` 只适用于地址在 `0x80000000` 附近的场景。

⚠️ 去掉这一行会怎样？链接时直接报 `relocation truncated to fit` 之类的错误。这是一个"看起来莫名其妙、其实原因很具体"的典型报错。

---

4. Makefile 精读：把 `make build` 拆开看

4.1 make 的心智模型：三样东西

`make` 的整个世界只有三样东西：

```
    目标 (target)  :  依赖 (prerequisites)
        └──────────────►  命令 (recipe)
```

翻译成人话：

> **"如果 `目标` 比 `依赖` 旧，或者 `目标` 不存在，就执行 `命令` 重新生成它。"**

就这一条规则，加上递归套用，构成了整个构建系统。举例：

```
$T/kernel: $(OBJS) $(linker) $U/initcode
    @$(LD) $(LDFLAGS) -T $(linker) -o $T/kernel $(OBJS)
```

读作：

> "`target/kernel` 依赖 `所有 .o 文件` + `链接脚本 qemu.ld` + `initcode`。其中任何一个比 `target/kernel` 新，就重新链接一次。"

🧠 **这个"比...新"的判断靠什么？ 靠文件修改时间戳（mtime）**。所以：

- `touch` 一个 `.c` 文件 → 它变"新"了 → 下次 `make` 会重编它
- 你电脑的**系统时间跳变**（比如虚拟机休眠后时间错乱） → make 判断失灵 → 出现"我明明改了却不重编"的怪现象。此时 `make clean` 从头再来。

🎯 **检验点 7**：`$(OBJS)` 里包含 `$K/main.o`，而 `$K/main.o` 又依赖 `$K/main.c`。那 `make build` 时 `main.c` 是怎么被编成 `main.o` 的？**Makefile 里找不到 `$K/main.o: $K/main.c` 这条规则**，它是从哪来的？

> 提示：`CFLAGS` 里有 `-MD`。查一下 gcc 的 `-MD` 选项生成什么文件，`.d` 文件里又写了什么。

4.2 逐段精读：`Makefile` 的六个区块

区块 1️⃣ 平台与模式开关（第 1~7 行）

```
#platform   := k210
platform    := qemu
mode := debug
#mode := release
K=kernel
U=xv6-user
T=target
```

变量	值	作用
`platform`	`qemu`	**决定整套代码走哪个分支**：用 `entry_qemu.o` 还是 `entry_k210.o`、用哪个链接脚本、加不加 `-D QEMU`
`mode`	`debug`	加不加 `-DDEBUG`
`K`/`U`/`T`	目录简写	`$K/main.o` = `kernel/main.o`

🧠 `#` **开头的行是注释。** 所以 `#platform := k210` 和 `#mode := release` 这两行是**被注释掉的备选值**。想切平台/切模式，就是**注释掉这行、把那行放出来**。

⚠️ **重要陷阱**：`platform := qemu` 和 `platform = qemu` 的区别是 `:=`（立即展开）vs `=`（延迟展开）。本 Makefile 混用了两种写法，在简单赋值下行为一致，但**在有相互引用的变量时差别巨大**。这是 Makefile 的经典考点。

`platform` 影响的三处：

```
# ① 用哪个入口汇编文件
ifeq ($(platform), k210)
   OBJS += $K/entry_k210.o
else
   OBJS += $K/entry_qemu.o        ← 我们走这条
endif

# ② 用哪个链接脚本（第 93~99 行）
ifeq ($(platform), qemu)
linker = ./linker/qemu.ld        ← 我们走这条
endif

# ③ 加不加 -D QEMU 宏（第 87~89 行）
ifeq ($(platform), qemu)
CFLAGS += -D QEMU                ← 我们走这条
endif
```

🧠 **`-D QEMU` 到底干了什么？** 它在**编译期**定义了一个叫 `QEMU` 的宏，于是代码里的 `#ifdef QEMU` 会走 true 分支。这就是 `kernel/printf.c` 里 `print_logo()` 有两套 ASCII art 的原因（`kernel/logo.c` 里还有一份几乎相同的代码，但它没被加进 `OBJS`，**不参与编译**——4.5 节的任务 A 会专门踩这个坑）：

```
#ifdef QEMU
void print_logo() { /* QEMU 版横幅 */ }
#else
void print_logo() { /* K210 版横幅 */ }
#endif
```

所以你改横幅时，要改的是 `#ifdef QEMU` 里面那一段，改 `#else`（K210）那段是白改的——**条件编译**就是这么"骗人"。这个坑 lab0 的动手任务 A 会亲自踩一次。

⌨️ **验证 `platform` 的影响（不动代码，只看展开）**：

```
make -n -B build | head -30  # -n = dry run 只打印不执行; -B = 视为全部过时（刚 build 过的话不加 -B 只会看到"无需做任何事"）
```

`make -n` 是你**理解任何 Makefile 的第一把钥匙**。它把 `$(OBJS)`、`$(CFLAGS)` 全部展开成最终命令行给你看。

🎯 **检验点 8**：`make -n build` 的输出里找到编译 `kernel/main.c` 的那一整行，数一数里面有几个 `-f` 开头的参数，并说出每一个的含义。

> 参考答案要点：`-ffreestanding`（不假设有标准库）、`-fno-common`（不同文件里的同名全局变量不合并）、`-fno-omit-frame-pointer`（保留栈帧指针，**gdb 才能正确回溯调用栈**）、`-fno-stack-protector`（不加栈保护，裸机没有运行时支持）。

区块 2️⃣ 编译选项（第 76~91 行）

```
CFLAGS = -Wall -Werror -Wno-error=infinite-recursion -O -fno-omit-frame-pointer -ggdb -g
CFLAGS += -MD
CFLAGS += -mcmodel=medany
CFLAGS += -ffreestanding -fno-common -nostdlib -mno-relax
CFLAGS += -I.
```

```
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-protector)

ifeq ($(mode), debug)
CFLAGS += -DDEBUG
endif

ifeq ($(platform), qemu)
CFLAGS += -D QEMU
endif

LDFLAGS = -z max-page-size=4096
```

逐条讲透（这是全章最有含金量的一段）：

选项	含义	为什么裸机内核需要它
`-Wall`	打开大部分警告	提前发现问题
`-Werror`	**把警告当成错误**	强制写干净代码；这也解释了为什么"有 warning 却 build 成功"不可能
`-Wno-error=infinite-recursion`	让警告不升级为错误，不阻挡程序继续执行	内核里有些函数**故意**写成形式上的无限递归/不返回
`-O`	开启优化	内核要性能
`-fno-omit-frame-pointer`	保留帧指针 `s0`/`fp`	**没有帧指针，gdb 就画不出调用栈**（backtrace 会断）
`-ggdb -g`	生成 gdb 用的调试信息	有了它 gdb 才能显示**变量名、函数名、行号**
`-MD`	生成 `.d` 依赖文件	记录头文件依赖（⚠️ 本 Makefile 没有 `include` 这些 `.d`，所以改头文件**不会**自动触发重编——改完 `.h` 记得 `make clean` 或 `touch` 对应的 `.c`）
`-mcmodel=medany`	代码位置无关的远地址寻址	内核链接在 `0x80200000`（见 3.4）
`-ffreestanding`	**我不在宿主操作系统里跑**	告诉编译器：不要假设有 `main` 特殊对待、不要假设有标准库可用
`-fno-common`	不同 `.c` 里的同名全局变量**不合并**	防止两个文件各自写了 `int x;` 却链接成同一个变量
`-nostdlib`	**链接时不链标准库**	裸机上没有 `libc`，`printf` 得自己写（就是 `kernel/printf.c`）

选项	含义	为什么裸机内核需要它
`-mno-relax`	禁止链接器做 RISC-V 的"松弛优化"	保持**机器码和汇编一一对应**，方便对照调试
`-I.`	把仓库根目录加入头文件搜索路径	所以代码里写 `#include "kernel/..."` 能找得到
`-fno-stack-protector`	不加栈保护	裸机没有 runtime 支持；这一行还用了 `$(shell ...)` **先试能不能加，能加才加**——这是很讲究的兼容写法
`-DDEBUG`	只在 `mode := debug` 时加	**控制调试打印的开关**

🧠 **`$(shell ...)` 是什么？** 它是 make 的**"执行一条 shell 命令，把输出当字符串用"**。上面那一行：

```
$(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-protector)
```

读法：**用交叉 gcc 试编一个空文件，加上 `-fno-stack-protector`；如果成功（`&&`），就输出 `-fno-stack-protector`，否则输出空串。** 这是一种**能力探测（feature probing）**——不硬编码，而是"问编译器你支持不支持"。

🎯 **检验点 9（核心）**：现在你能解释 lab0 和 lab1 启动输出的差别了。

```
lab0 (mode := debug):       lab1 (mode := release):
  ...logo...                    ...logo...
  kernel_end: 0x802XXXXX, phystop: 0x80600000   ← 没有这两行!
  kinit
  hart 0 enter main()...        ← 也没有这行
```

问题：这三行输出分别由哪段代码产生？为什么 lab1 没有？

> 提示：去 `kernel/kalloc.c` 和 `kernel/main.c` 找 `#ifdef DEBUG`。
> 答：`kalloc.c:36-39` 的 `printf("kernel_end: ...")` / `printf("kinit\n")` 和 `main.c` 的 `printf("hart %d enter main()...")` 都被 `#ifdef DEBUG` 包着；lab0 的 `mode := debug` 加了 `-DDEBUG`，lab1 的 `mode := release` 没加。

区块 3️⃣ 链接规则（第 101~108 行）

```
# Compile Kernel
$T/kernel: $(OBJS) $(linker) $U/initcode
    @if [ ! -d "./target" ]; then mkdir target; fi
    @$(LD) $(LDFLAGS) -T $(linker) -o $T/kernel $(OBJS)
    @$(OBJDUMP) -S $T/kernel > $T/kernel.asm
    @$(OBJDUMP) -t $T/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $T/kernel.sym

build: $T/kernel userprogs
```

三行核心命令，值得逐个理解：

命令	作用
`ld -z max-page-size=4096 -T linker/qemu.ld -o target/kernel $(OBJS)`	**链接**：按 `qemu.ld` 的布局规定，把一堆 `.o` 拼成 ELF 可执行文件 `target/kernel`
`objdump -S target/kernel > target/kernel.asm`	**反汇编**：`-S` 表示"汇编和 C 源码交替显示"。**这份文件是你读内核最有力的武器**
`objdump -t target/kernel | sed ... > target/kernel.sym`	**抽符号表**：把每个函数/全局变量的**名字和地址**整理成两张干净的列

🧠 `@` **前缀是什么？** 表示"**执行这条命令，但不要把命令本身打印出来**"。所以 `make build` 时你看不到那一长串 `ld ...`，输出很干净。想看到的话，`make -n` 或者临时把 `@` 去掉。

🧠 `kernel.asm` **和** `kernel.sym` **怎么用？**

- 想找 `main` 函数被放在哪个地址？`grep " main$" target/kernel.sym`
- 想找某条指令对应哪行 C 代码？在 `kernel.asm` 里搜函数名
- 内核 panic 打出 `0x80201234`？在 `.sym` 里找**小于且最接近** `0x80201234` 的地址，就是出事的函数

⚠️ `sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d'` **在干什么？** 三段 sed 脚本用分号隔开：

1. `1,/SYMBOL TABLE/d` —— 从第 1 行删到含 `SYMBOL TABLE` 的行（去掉 objdump 的头部）
2. `s/ .* / /` —— 把行中间多余的空格压缩（把 `addr  flags  section  name` 压成 `addr name`）
3. `/^$$/d` —— 删掉空行（`$$` 在 Makefile 里转义成 sed 眼里的 `$`，即行尾）

这就是"用一个命令把原始输出变成能用的小文件"的实际工程手法。 你以后分析日志时会经常写这种小管道。

区块 4️⃣ 用户程序规则（第 155~213 行）

```
$U/initcode: $U/initcode.S
    $(CC) $(CFLAGS) -march=rv64g -nostdinc -I. -Ikernel -c $U/initcode.S -o $U/initcode.o
    $(LD) $(LDFLAGS) -N -e start -Ttext 0 -o $U/initcode.out $U/initcode.o
    $(OBJCOPY) -S -O binary $U/initcode.out $U/initcode
    $(OBJDUMP) -S $U/initcode.o > $U/initcode.asm
```

🧠 `initcode` **是什么？为什么它不是普通的用户程序？**
它是内核**内置**的一小段机器码——内核没法"从磁盘加载第一个程序"，因为那时还没有文件系统可用。所以 xv6 把这段启动代码**编成纯二进制，硬编码进内核镜像**（在 `proc.c` 里就是一个 `initcode[]` 字节数组）。

看这四步：

1. 汇编 `initcode.S` → `initcode.o`
2. 链接，`-Ttext 0` —— 让代码从**虚拟地址 0** 开始（这就是"第一条用户指令在地址 0"的由来！）
3. `objcopy -O binary` —— 从 ELF 格式**剥成裸二进制**（去掉 ELF 头，只留机器码字节）
4. 反汇编存档

⚠️ `-S` **在这两步里含义完全不同！**

- `objcopy -S` = **strip**，去掉符号信息

- `objdump -S` = **source intermix**，显示源码和汇编交替

同一个字母，两个工具，两种意思。这是新手最容易混淆的一处。

```
ULIB = $U/ulib.o $U/usys.o $U/printf.o $U/umalloc.o

_%: %.o $(ULIB)
    $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $@ $^
    $(OBJDUMP) -S $@ > $*.asm
    $(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $*.sym

$U/usys.S : $U/usys.pl
    @perl $U/usys.pl > $U/usys.S
```

🧠 `_%: %.o $(ULIB)` **里的** `%` **是什么？** 这是**模式规则（pattern rule）**。`%` 是通配符，代表"任意词干"。所以这一条规则同时适用于：

```
xv6-user/_halt      ← 由 xv6-user/halt.o 生成
xv6-user/_ls        ← 由 xv6-user/ls.o 生成
xv6-user/_cat       ← 由 xv6-user/cat.o 生成
...（一条规则管 17 个用户程序）
```

这就是 Makefile 的威力，也是它的难点：一条规则，一次写完，处处生效。🧠 `$@` = 目标名（`_halt`），`$^` = 全部依赖，`$*` = 匹配到的词干（`halt`）——这三个是**自动变量**，是写模式规则的关键。

🧠 **为什么每个用户程序都要链** `$(ULIB)`**？** 因为 `ulib.o` 提供 `printf`、`strlen` 等基础函数；`usys.o` 提供**全部系统调用的用户态桩**（`fork`、`read`、`write`…）；`umalloc.o` 提供堆内存分配。没有它们，用户程序连 `printf` 都用不了。

🧠 `$U/usys.S : $U/usys.pl` **+** `perl` **这一步的深意**：用户程序的每个系统调用都需要一小段汇编桩（把调用号放进 `a7`，执行 `ecall`）。手写 27 个函数太啰嗦，所以用 **perl 脚本生成**。这体现了**"用代码生成代码"（metaprogramming）** 的思想——你会在这里第一次看到它。

```
usys.pl 里的 entry("halt")  ──perl──▶  usys.S 里的
                                        .global halt
                                        halt:
                                          li a7, SYS_halt    ← 调用号进 a7
                                          ecall              ← 陷入内核
                                          ret                ← 返回
```

区块 5️⃣ 目标列表与文件系统镜像（第 189~232 行）

```
UPROGS=\
    $U/_init\
    $U/_sh\
    $U/_cat\
    ...
    $U/_mv\
```

🧠 **`UPROGS` 就是"要装进文件系统镜像的用户程序清单"。** 想加一个新命令（比如 lab1 的 `halt`），就要在这里加一行 `$U/_halt\`。
⚠️ **这一行的反斜杠 `\` 不能少**，它表示"这一行没结束，下一行还是同一条语句"。加漏了会报 `missing separator`。

```
fs: $(UPROGS)
    @if [ ! -f "fs.img" ]; then \
        echo "making fs image..."; \
        dd if=/dev/zero of=fs.img bs=512k count=512; \
        mkfs.vfat -F 32 fs.img; fi
    @sudo mount fs.img $(dst)
    @if [ ! -d "$(dst)/bin" ]; then sudo mkdir $(dst)/bin; fi
    @sudo cp README $(dst)/README
    @for file in $$( ls $U/_* ); do \
        sudo cp $$file $(dst)/$${file#$U/_};\
        sudo cp $$file $(dst)/bin/$${file#$U/_}; done
    @sudo umount $(dst)
```

`fs` 目标做了什么（这一步非常关键）：

```
① 如果 fs.img 不存在 → dd 造一个 256MB 的空文件 → mkfs.vfat 格式化成 FAT32
                                    ↓
② sudo mount fs.img /mnt      —— 把这个"磁盘镜像文件"当磁盘挂载到 /mnt
                                    ↓
③ sudo cp xv6-user/_ls /mnt/ls && sudo cp xv6-user/_ls /mnt/bin/ls
   （循环把 17 个用户程序拷进去，同时拷到根目录和 /bin/）
                                    ↓
④ sudo umount /mnt            —— 卸载，镜像文件里就有了内容
```

🧠 **`${file#$U/_}` 是什么魔法？** 这是 shell 的**前缀剥离**语法 `#`：把变量 `file` 里**开头的** `xv6-user/_` 去掉。

- `file = xv6-user/_halt` → `${file#xv6-user/_}` = `halt`
- 目的：把带下划线的构建产物名 `_halt` 转成**用户看到的命令名** `halt`

⚠️ **为什么要 `sudo`？** 因为 `mount` 是特权操作（要操作系统的 VFS 层）。**这是 `make fs` 必须在有 `sudo` 权限的环境里跑的原因**，也是**为什么改用户程序后必须重跑 `make fs`**。

💡 **预装虚拟机的小优化（可选）**：如果你嫌每次 `sudo` 麻烦，可以在虚拟机里给 `mount`/`umount` 配免密 sudo；或者在课程提供的环境里 `fs.img` **已经预生成好**，此时 `make fs` 会直接进入 mount 步骤。**这不影响本实验的任何学习目标。**

⚠️ **`$$file` 为什么是两个 `$`？**
Makefile 里 `$` 是 make 自己的变量前缀。要让 `$` 传给 shell，就得**写两次**：`$$` → shell 收到 `$`。
所以 `$$( ls $U/_* )` 里的 `$$(` 是 shell 的命令替换，而 `$U/_*` 是 make 展开成 `xv6-user/_*`。**两套变量系统混在一行里，这是 Makefile 最容易看晕的地方。**

区块 6️⃣ 运行与清理（第 128~153、243~251 行）

```
ifndef CPUS
CPUS := 2
endif

QEMUOPTS = -machine virt -kernel $T/kernel -m 128M -nographic
QEMUOPTS += -smp $(CPUS)
QEMUOPTS += -bios default
QEMUOPTS += -drive file=fs.img,if=none,format=raw,id=x0
QEMUOPTS += -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

run: build
else 分支:
    @$(QEMU) $(QEMUOPTS)
```

逐个 QEMU 参数讲透：

参数	含义	教学价值
`-machine virt`	用 QEMU 的**通用虚拟平台** `virt`	⚠️ **这个平台的内存从** `0x80000000` **开始**——直接决定了内核为什么被放在 `0x80200000`（第 5 章）
`-kernel target/kernel`	把内核 ELF **当作固件加载**	QEMU 会读 ELF 头，按 ELF 里规定的地址装载
`-m 128M`	给虚拟机 **128MB 物理内存**	所以物理内存范围是 `0x80000000` ~ `0x80000000 + 128M` = `0x88000000`，但内核只用到 `0x80600000`（`PHYSTOP`）——见第 5 章
`-nographic`	**不要图形窗口**，把串口接到当前终端	这也是为什么退出要用 `Ctrl+A X`
`-smp 2`	模拟 **2 个 CPU 核**（2 个 hart）	解释了 `_entry` 里为什么要按 `a0`（hartid）分配不同栈
`-bios default`	用 QEMU **自带的 OpenSBI 固件**	所以**不需要**编译仓库里的 RustSBI
`-drive file=fs.img,...`	把 `fs.img` 挂成一块 **virtio 磁盘**	内核的 `virtio_disk.c` 驱动它，读出的就是 FAT32 文件系统
`-device virtio-blk-device,...`	显式指定 virtio-mmio 总线上挂块设备	与 `memlayout.h` 里的 `VIRTIO0` 地址对应

🎯 **检验点 10（重要）**：`-m 128M` 给出 128MB 内存，但内核只管理到 `0x80600000`。这两个数字能对上吗？

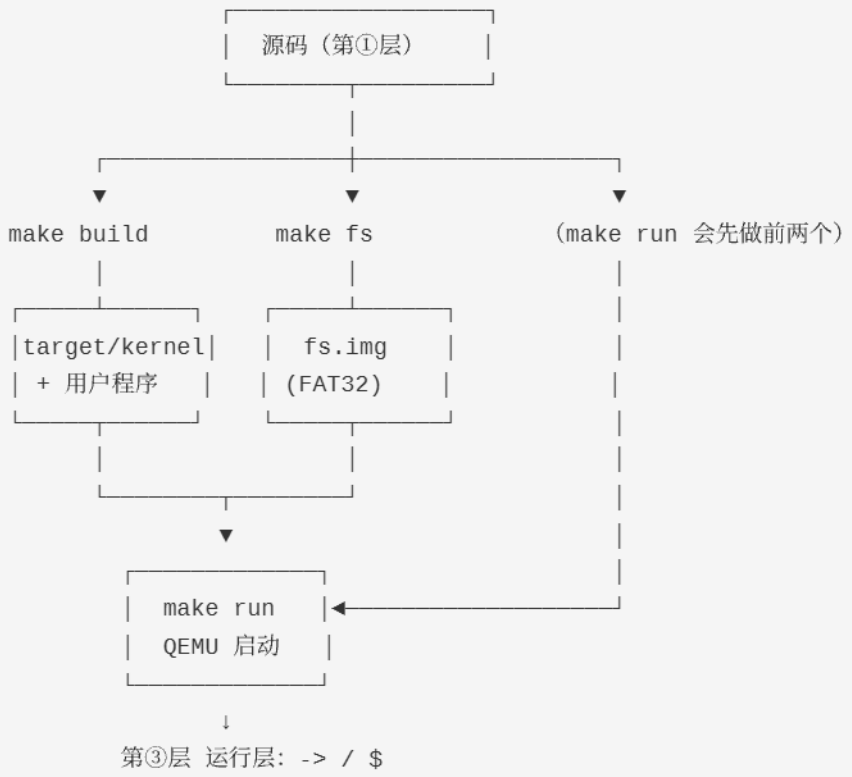
> 算一下：`0x80600000 - 0x80000000` = `0x00600000` = **6MB**。
> **所以：QEMU 给了 128MB，但 xv6 只用最前面的 6MB。** 为什么？（第 5 章和 lab1 会给出完整答案：`PHYSTOP` 是内核自己写死的常量，`0x80200000`~`0x80600000` 只给了 4MB 可用物理页，后面 122MB 内核压根没碰。）

```
clean:
    rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
    */*.o */*.d */*.asm */*.sym \
    $T/* \
    $U/initcode $U/initcode.out \
    $K/kernel \
    .gdbinit \
    $U/usys.S \
    $(UPROGS)
```

💡 **`clean` 里也删 `.gdbinit`**——这是从原版 xv6 继承的写法。本仓库的 `make gdb` 并**不**生成 `.gdbinit`（连接参数由 `-ex` 直接给出）；动手任务 C 带你把这两个目标读懂。

⚠️ **`clean` 不删 `fs.img`**。这是一个有意的设计：`fs.img` 重新生成很慢（要 `dd` 256MB + `mkfs.vfat` + `sudo mount`），而且里面的内容就是用户程序的新版本。如果你怀疑镜像坏了，手动 `rm fs.img`。

4.3 一张图看懂三个 make 目标



4.4 `make build` / `make fs` / `make run` 各管什么（必背）

目标	依赖	管什么	什么时候必须重跑
`make build`	`target/kernel`、`userprogs`	编译内核 + 全部用户程序	改了任何 `.c` / `.h` / `.S` / `Makefile` / `.ld` 之后
`make fs`	`$(UPROGS)`	生成/刷新 `fs.img`（需 sudo）	改了 `xv6-user/` 下任何文件之后

目标	依赖	管什么	什么时候必须重跑
`make run`	`build`	启动 QEMU	想跑的时候（⚠️ **不会**自动 `fs`）
`make clean`	—	删构建产物（保留 `fs.img`）	出现莫名奇妙的"改了没生效"时
`make -n <目标>`	—	**只打印命令，不执行**	**理解 Makefile 的神器，随时可用**

🎯 **检验点 11**：Makefile 里写的是 `run: build`。如果你只敲 `make run`，`make` 会**自动先做** `build` 吗？会顺手把 `fs` 也做了吗？

> `build` 会——这正是 make 的核心能力：**递归满足依赖**。但 `fs` **不会**：`run` 只依赖 `build`，所以改过 `xv6-user/` 之后必须自己先 `make fs`，否则 QEMU 跑的还是旧 `fs.img`。这就是 2.2 节那个"改了没生效"坑的根源。

4.5 ✍️ 动手任务 A：改启动横幅，验证完整闭环

目的：亲手走通「改源码 → 重新构建 → 重新打包 → 重跑 → 看到变化」这条闭环。这是 lab0 最核心的手感训练。

步骤：

⌨️ ① 打开 `kernel/printf.c`，找到 `print_logo()` 函数——**真正的启动横幅在这里**（约第 153 行起，`#ifdef QEMU` 分支内；⚠️ 不是 `#else` 那段）。

> ⚠️ **别改错文件**：仓库里还有一个 `kernel/logo.c`，内容和 `print_logo()` 几乎一样，但它**没有被加进 Makefile 的 `OBJS`**，改它不会生效（用 `make -n -B build | grep logo` 验证：找不到任何编译它的命令）。有兴趣可以先改 `logo.c` 跑一次 `make run` 亲身体验"死代码"这个坑，再来改 `printf.c`。

⌨️ ② 把函数里那一串 `printf` 换成你自己的横幅。要求：

- 至少包含**你的学号后 4 位**和 **"lab0"** 字样
- 每个 `printf` 字符串末尾必须有 `\n`
- 保留 `#ifdef QEMU` / `#else` / `#endif` 结构不动

参考写法（只是示例，写你自己的）：

```
#ifdef QEMU
void print_logo() {
    printf("\n");
    printf("  ============================================\n");
    printf("   xv6-k210  |  lab0  |  学号 2025xxxx\n");
    printf("   build OK - environment verified\n");
    printf("  ============================================\n");
    printf("\n");
}
#else
```

⌨️ ③ 重新构建并运行：

```
make run
```

⌨️ ④ **观察三件事**：

1. 你的横幅出现了吗？
2. 横幅**下面**跟了哪些内核初始化打印？逐行抄下来。
3. 其中有一行形如 `kernel_end: 0x802XXXXX, phystop: 0x80600000`——**记下这两个数字**，动手任务 B 要用。

⚠️ **如果横幅没变，按这个顺序排查（不要直接问 AI 要答案，先自己走一遍）**：

```
① 我改的是 #ifdef QEMU 里那段吗?      → 改错分支是最常见的原因
② 我保存文件了吗?                      → 编辑器里看有没有"未保存"标记
③ 我跑的是 make run 还是只跑了 make build?
   · 只跑 build → 内核是新的，但 QEMU 没重启 → 重跑 make run
④ make 有没有真的重编 printf.c?
   · 用 make -n -B build | grep printf 看命令行展开
   · 或者 make run 的输出里有没有出现对 printf.c 的编译命令
⑤ 时间戳问题?
   · make clean && make run 强制全量重建
```

🎯 **任务 A 的检验点**：用一句话解释——**为什么"改 `kernel/printf.c` "不需要重跑 `make fs`？**

> 答：因为 `printf.c` 属于**内核**（编进 `target/kernel`），不属于**用户程序**（装进 `fs.img`）。`fs.img` 里只放 `xv6-user/` 下的东西。**内核和用户程序是两条独立的产物链。**
> 追问：那为什么改 `kernel/logo.c` 连重编都不用等？——因为它根本不在 `OBJS` 里，是死代码。
> → 这就是"三层世界观"的第一次实战应用。

---

5. 链接脚本与外设地址：内核被放在哪里

5.1 打开链接脚本：`linker/qemu.ld`

```
OUTPUT_ARCH(riscv)
ENTRY(_entry)                          ← ①

BASE_ADDRESS = 0x80200000;             ← ②

SECTIONS
{
    . = BASE_ADDRESS;                  ← ③
    kernel_start = .;
    . = ALIGN(4K);
    text_start = .;
    .text : {
        *(.text .text.*)
        . = ALIGN(0x1000);
        _trampoline = .;               ← ④
        *(trampsec)
        . = ALIGN(0x1000);
```

```
        ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
        PROVIDE(etext = .);
    }
    ...
    PROVIDE(kernel_end = .);         ← ⑤
}
```

逐行讲：

位置	含义
① `ENTRY(_entry)`	规定**程序的入口符号**是 `_entry`。ELF 头里会记录"从这里开始执行"。🧠 它**不等于** C 的 `main`——真正的第一条指令是汇编写的 `_entry`（在 `kernel/entry_qemu.S`）
② `BASE_ADDRESS = 0x80200000`	内核被放到 **物理地址 0x80200000 = 2GB + 2MB** 处
③ `. = BASE_ADDRESS;`	🧠 `.` 是**位置计数器**（location counter），表示"接下来要放的地址"。这一句把它设成 `0x80200000`
④ `_trampoline` + `ASSERT`	强制 **trampoline 代码必须正好占 1 页（0x1000 = 4096 字节）**。这是 lab1 的核心机制之一，见下
⑤ `kernel_end`	内核的**结束地址**，会在 `kalloc.c` 里被用来"从哪开始可以分配物理页"

5.2 为什么是 `0x80200000`？—— 一次地址考古

```
0x0000_0000  ┌──────────────────────────────────────────┐
             │  空 / 未映射                             │
0x0200_0000  ├──────────────────────────────────────────┤
             │  CLINT（定时器/核间中断）   0x02000000   │ ← memlayout.h
0x0C00_0000  ├──────────────────────────────────────────┤
             │  PLIC（外部中断控制器）     0x0c000000   │ ← memlayout.h
0x1000_0000  ├──────────────────────────────────────────┤
             │  UART0（串口）              0x10000000   │ ← memlayout.h
0x1000_1000  ├──────────────────────────────────────────┤
             │  VIRTIO0（磁盘）            0x10001000   │ ← memlayout.h
             ├──────────────────────────────────────────┤
             │  （地址空洞）                            │
0x8000_0000  ├──────────────────────────────────────────┤
             │  OpenSBI 固件（M 模式）     0x80000000   │ ← RUSTSBI_BASE
             │  ↑ QEMU -machine virt 的内存起点         │
0x8020_0000  ├──────────────────────────────────────────┤
             │  ★ xv6 内核从这里开始       KERNBASE     │
             │    .text / .rodata / .data / .bss        │
0x8060_0000  ├──────────────────────────────────────────┤
             │  PHYSTOP —— xv6 管理的物理内存上界       │
             │  （再往上是 128M 里的剩余部分，xv6 不用）│
0x8800_0000  ├──────────────────────────────────────────┤
             │  QEMU -m 128M 的内存上界                 │
```

```
0xFFFF_FFFF └──────────────────────────────────────────────────┘
```

这个"考古"分三步：

第 1 步：`0x80000000` 从哪来？
`QEMUOPTS` 里的 `-machine virt`。QEMU 的 `virt` 虚拟平台规定：**DRAM 从 `0x80000000` 开始**。这是一个**平台约定**，写死在 QEMU 的 `hw/riscv/virt.c` 里。

第 2 步：为什么内核不在 `0x80000000` 开始，要往后挪 2MB？
因为**那段地址被 OpenSBI 占了**。开机流程是：

```
QEMU 加电（PC = 0x1000）
   ↓
OpenSBI 固件（被 QEMU 按 -bios default 装载）在 0x80000000 开始运行
   ↓
OpenSBI 做 M 模式初始化（配时钟、中断、串口…）
   ↓
OpenSBI 跳到内核入口 _entry（在 0x80200000）
```

所以 `0x80000000 ~ 0x80200000`（2MB）是**留给固件的**，内核从 `0x80200000` 开始，**两边不打架**。

第 3 步：`0x80600000` 从哪来？
这是 xv6 **自己写死的常量**，在 `kernel/include/memlayout.h` 里：

```
#define KERNBASE    0x80200000L
#define PHYSTOP     0x80600000L
```

🎯 **检验点 12（关键计算题）：**

- 内核可用的物理内存区间是 `KERNBASE` ~ `PHYSTOP`，一共多大？
- 按 4096 字节一页算，一共多少页？
- 其中**已经给内核代码/数据用掉**的部分大约多少？（看启动打印的 `kernel_end`）
- 所以**真正能给进程分配的页**大约多少页？

> 参考：`0x80600000 - 0x80200000` = `0x400000` = **4MB** = **1024 页**。
> 内核自身（`kernel_end - 0x80200000`）大约 100~200KB，即 25~50 页。
> 所以可用约 **970~1000 页 ≈ 4MB**。
> 💡 **这就是动手任务 A 里那两个数字的意义**：`kernel_end` 是"内核用到哪"，`phystop` 是"分配器管到哪"，**两者之间就是空闲物理页**。

5.3 `_trampoline` 的 `ASSERT` 为什么重要

```
. = ALIGN(0x1000);
_trampoline = .;
*(trampsec)
. = ALIGN(0x1000);
ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
```

🧠 **`trampsec` 是什么？** 是 `kernel/trampoline.S` 里声明的**自定义段**：

```
.section trampsec
.globl trampoline
trampoline:
```

🧠 **`trampoline`（蹦床）在这里指什么？** 一段**在用户页表和内核页表里映射到同一个虚拟地址**的代码。它负责"用户态陷入内核时切换页表、保存寄存器"。lab1 会讲透，lab0 只要记住一件事：

> **它必须正好 1 页。** 因为它的虚拟地址是 `MAXVA - PGSIZE`（最高的一页），如果超过 1 页就会**越过地址空间顶点**。

那个 `ASSERT` 就是**链接期守卫**：一旦有人往里加了代码导致超页，**链接直接失败**，而不是等到运行时神秘崩溃。

🎯 **检验点 13**：`ASSERT` 是**链接期**检查还是**运行期**检查？

> **链接期。** 这是"尽早失败（fail fast）"的工程原则：能在构建时发现的错误，绝不拖到运行时。你在 lab1 会看到很多这种设计（`panic()` 也是同一个思路）。

5.4 读产物：`kernel.asm` 与 `kernel.sym`

构建完之后，`target/` 目录里有三个宝：

⌨️ **看符号表，找到入口和关键函数**：

```
grep -E " _entry$| main$| kernel_end$| etext$| trampoline$" target/kernel.sym
```

你会看到类似（地址是示意，以你的实际输出为准）：

```
0000000080200000 _entry
0000000080200020 main
0000000080230000 _trampoline
0000000080240000 etext
0000000080260000 kernel_end
```

🧠 **注意 `_trampoline` 的地址是 8 的倍数但 `ASSERT` 要 0x1000 对齐**——因为 `ALIGN(0x1000)` 保证它从页边界开始，这里 0x80230000 正好是 0x1000 的整数倍（0x...30000 % 0x1000 == 0），✅ 对上了。

⌨️ **看反汇编，对照 C 源码**：

```
# 只看 main 函数的前 20 行汇编（-A 20 = 匹配行及其后 20 行；
# 不要用 awk '/<main>:/,/^$/' 这种"到空行止"的范围写法——objdump -S 在标签后紧跟空行，会只吐出一行）
grep -A 20 '<main>:' target/kernel.asm
```

`objdump -S` 的好处是**汇编和 C 源码交替出现**：

```
kernel/main.c:28
int main(int hartid, uint64 dtb_pa){
    80200020:   fe010113    addi  sp,sp,-32      ← 开辟栈帧
    ...
```

🧠 **这就是"读内核"的正确姿势**：不是硬背汇编，而是**让编译器告诉你"这行 C 变成了什么机器码"**。当你不理解某个 C 语句的效果时，去看 `kernel.asm`，比看任何教程都准。

⚠️ `target/kernel.asm` **可能非常大**（几 MB）。不要用编辑器全量打开，用 `grep` / `awk` / `less` 加搜索。

5.5 ✍️ 动手任务 B：读懂启动时那两个数字

目的：把"物理内存布局"从一张图变成**你自己实测出来的数字**。

⌨️ ① `make run`，在启动输出里找到这一行（由 `kernel/kalloc.c:37` 打印）：

```
kernel_end: 0x802XXXXX, phystop: 0x80600000
```

⌨️ ② **记下来，然后填空计算**（用计算器或 python 都行）：

```
python3 - <<'EOF'
kernel_end = 0x80200000 + 0x00000000     # ← 把 0x802XXXXX 写进来（删掉 + 0x00000000）
KERNBASE   = 0x80200000
PHYSTOP    = 0x80600000
PGSIZE     = 4096

kernel_size  = kernel_end - KERNBASE
total_region = PHYSTOP - KERNBASE
free_pages   = (PHYSTOP - kernel_end) // PGSIZE

print(f"内核自身大小      : {kernel_size} 字节 = {kernel_size/1024:.1f} KB =
{kernel_size//PGSIZE} 页")
print(f"内核管理的内存区间: {total_region} 字节 = {total_region/1024/1024} MB =
{total_region//PGSIZE} 页")
print(f"可分配给进程的空闲: {free_pages} 页 = {free_pages*PGSIZE/1024/1024:.2f} MB")
EOF
```

⌨️ ③ **验证你的计算**（⚠️ 本课程包的 `xv6-user/` 里**没有** `sysinfo` 用户程序，这一步**可跳过**，以笔算结果为验收依据）：如果你之后自己实现了 `sysinfo` 系统调用（MIT xv6 课程经典练习），在 shell 里跑 `sysinfo`，它报的 `freemem` 和你算的 `free_pages * 4096` 应该是**同一个量级**——差值来自已经分配出去的页表页、进程结构、内核栈等等，这是正常的。

🎯 **任务 B 的检验点**：用一句话回答——**QEMU 给了 128MB 内存，为什么 xv6 只用 4MB?**

> 答：因为 `PHYSTOP` 是 xv6 **源码里写死的常量** `0x80600000`。`kinit()` 只把 `kernel_end ~ PHYSTOP` 之间的页放进空闲链表，剩下的 122MB **地址上存在，但分配器不知道它的存在**。
> → 这是"**机制与策略分离**"的一个早期伏笔：物理内存的**边界是策略**（写死的常量），分配器的**机制**（空闲链表）与边界无关，所以改边界只需改一个宏。

---

6. gdb 调试：让内核停下来给你看

6.1 为什么必须学会调试内核

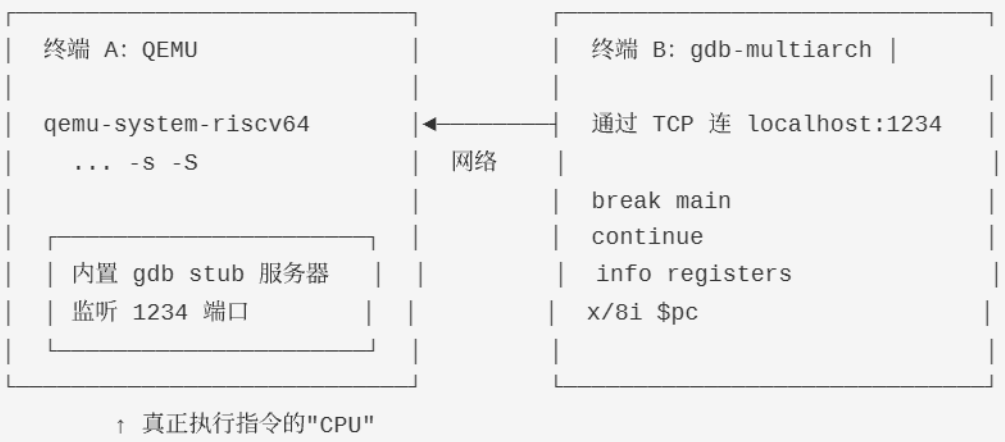
前面所有东西你都能靠"读代码 + 打印"搞定。但有一类问题只能靠调试器：

问题	printf 够吗	gdb 怎么帮你
内核在 `panic("kfree")` 挂了，我想知道**谁**在乱释放内存	❌	断在 panic 前的调用栈，一层层回溯
我想看**启动第一条指令时** `sp` 是什么	❌	在内核第一条指令处停下，`info registers`
我想看某个地址里的内存内容	勉强	`x/16gx 0x80200000` 直接看
我想单步执行 C 代码，看每行执行后变量怎么变	❌	`next` / `step`

一句话：gdb 是"能暂停时间"的显微镜。

6.2 gdb 的原理：为什么它能"停住" CPU

这是一个非常好懂、但很多人从没想明白的问题。



三个部件各管什么：

部件	作用
QEMU 的 `-s`	打开**内置的 gdb stub**（一个迷你调试服务器），监听 TCP **1234** 端口
QEMU 的 `-S`	**启动后立刻暂停**，等 gdb 来说"继续"。没有 `-S` 的话内核会一路跑到底，你根本来不及下断点
gdb 侧的 **符号表**	gdb 需要知道"`main` 这个函数在地址 X、变量 `hartid` 在栈上偏移 Y"。这些信息**不在内存里**，而在 `target/kernel` 这个 ELF 里（因为编译时加了 `-g -ggdb`）
gdb 侧的 `target remote :1234`	让 gdb 不去启动新进程，而是**接管远端一个已经在跑的 CPU**

🧠 **"远程调试"里的"远程"是什么意思？** 不是说"在另一台电脑上"——**本机也可以**。它指的是**调试协议是分层的**：gdb（前端）↔ 调试协议（GDB Remote Serial Protocol）↔ 被调试目标（stub）。这个协议是文本化的，你甚至能手动用 `nc` 说它：

```
# 手工向 gdb stub 发一条协议命令，读出所有寄存器的值（十六进制）
printf '$g#67' | nc localhost 1234
```

（若报 `nc: 未找到命令`，说明虚拟机没装 netcat：`sudo apt install -y netcat` 装上再试，或直接跳过——这条只是帮你体会"协议是文本化的"，不属于验收项。）

🧠 **为什么这种设计很牛？** 因为**同一个 gdb 前端**可以接：本机进程、QEMU 里的虚拟机、一台上位机连着开发板、甚至云端容器。**前端与后端解耦**——这就是好的架构。

6.3 ✍️ 动手任务 C：读懂 Makefile 里的 gdb 工作流

⚠️ **注意**：在开始之前，先确认 gdb 叫什么：

```
gdb-multiarch --version        # 如果这条能跑，就用它
```

⌨️ **步骤 ①：打开 `Makefile`，找到 `run:` 目标（约第 143 行）后面跟着的 `qemu-gdb:` 和 `gdb:` 两个目标（约第 154~170 行）**，逐行读懂。如果你的 Makefile 版本里没有这两段，再把下面的内容补到 `run:` 之后：

```
# ============================================================
# 新增: gdb 调试工作流
# ============================================================

# 终端 A 跑这个: 启动 QEMU, 但开机就暂停, 等 gdb 连接
#   -s  : 打开内置 gdb stub, 监听 TCP 1234
#   -S  : 启动后立刻暂停（S = Stop）
qemu-gdb: build
    @echo "*** QEMU 已暂停, 等 gdb 连接 1234 端口 ***"
    @echo "*** 请另开一个终端, 运行: make gdb ***"
    @$(QEMU) $(QEMUOPTS) -s -S

# 终端 B 跑这个: 用 gdb-multiarch 连上 QEMU（不生成 .gdbinit, 连接参数由 -ex 直接给出）
gdb: $T/kernel
    gdb-multiarch target/kernel -ex 'set arch riscv:rv64' -ex 'target remote localhost:1234'
```

⚠️ **Makefile 的潜规则**：recipe（命令）行的开头**必须是真正的 TAB 字符**，不能是空格。用编辑器时打开"显示不可见字符"确认。`missing separator` **报错 99% 是这个原因。**

⌨️ **步骤 ②：两个终端，按顺序操作**

终端 A：

```
make qemu-gdb
```

终端 B（另开一个）：

```
make gdb
```

终端 B 里你应该看到：

```
GNU gdb (GDB) ...
Reading symbols from target/kernel...
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()          ← 注意！PC 还在 0x1000
(gdb)
```

🧠 **为什么 PC 是 `0x1000` 而不是 `0x80200000`？** 因为**真正执行的第一条指令不是内核的**！`0x1000` 是 QEMU 的**复位向量（reset vector）**——CPU 刚加电时 PC 被硬件设成 `0x1000`，那里是 **OpenSBI 固件**的入口。

这是一个极其重要的认知：内核不是"第一个跑的东西"。前面还有固件。这直接引出 lab1 的第一个专题。

⌨️ **步骤 ③：亲手做四个实验**

```
# 实验 1：看当前寄存器状态（应该在 OpenSBI 里）
info registers
# 观察 pc 的值；观察 a0（它应该是 hartid）

# 实验 2：在 xv6 内核入口 _entry 处下断点，然后继续
break *_entry
continue
# 现在你停在 xv6 的第一条指令上了！
#   ▸ 抄下此时的 pc 值，应该是 0x80200000
#   ▸ 注意要写 *_entry（按地址下断点）：写 break _entry 会停到 0x80200004（+4 位置）
#   ▸ 抄下此时的 sp 值—它是多少？有意义吗？

# 实验 3：看 _entry 的 5 条指令
x/5i $pc
# 应该看到（对照 kernel/entry_qemu.S）：
#   add  t0,a0,1
#   slli t0,t0,14
#   la   sp,boot_stack
#   add  sp,sp,t0
#   call main

# 实验 4：单步执行这 5 条，观察 sp 和 t0 怎么变
stepi                          # 执行 1 条机器指令
info registers t0 sp           # 看 t0 和 sp
stepi
info registers t0 sp
stepi
info registers t0 sp
stepi
info registers t0 sp
# ▸ 重点：第 3 条 `la sp, boot_stack` 之后 sp 变成多少？
# ▸ 第 4 条 `add sp, sp, t0` 之后 sp 又变成多少？
# ▸ 这个值和你预估的 boot_stack 地址对得上吗？
```

🎯 **任务 C 的检验点（核心理解）：**

1. **`_entry` 处 `sp` 的值有意义吗？**

   > **没有意义。** 因为还没有栈——`_entry` 干的第一件事就是**建立栈**。所以前两条指令 `add t0, a0, 1` /

> `slli t0, t0, 14` **绝对不能碰栈**（不能 push/pop，不能调用函数）。
> 💡 **这就是"为什么开头是这两条奇怪的指令"的答案**：它们只在寄存器间运算，**不需要栈**。

2. **`slli t0, t0, 14` 里的 `14` 是怎么来的？**

> `1 << 14 = 16384 = 4 页 × 4096 = 16KB`。每个 hart 分 16KB 栈。看 `entry_qemu.S` 末尾的 `.space 4096 * 4 * 2`——一共 32KB，**正好够 2 个 hart 各 16KB**。
> 所以 `-smp 2` 和 `.space 4096*4*2` **是必须匹配的一对**。如果 `-smp 4` 就溢出爆栈了。

3. **`_entry` 里 `sp` 拿到的是物理地址还是虚拟地址？**

> **物理地址。** 因为那时页表还没开（`satp` 还是 0）。`kvminithart()` 是在 `main()` 里才调用的。
> → 这直接引出 lab1 的核心专题：**什么时候地址是物理的，什么时候变成虚拟的。**

4. **`call main` 之后会怎样？如果 `main` 真的 `return` 了？**

> 会跳到 `loop: j loop`，**死循环**。因为 `main()` 正常情况**永远不返回**（它最后调用 `scheduler()`，进调度器就不回来了）。`j loop` 是**兜底保险**——万一真返回了，至少不会跑飞乱执行内存。

6.4 gdb 常用命令速查（贴墙上）

命令	简写	作用
`break <位置>`	`b`	下断点。位置可以是函数名 `main`、`file:line`、地址 `*0x80200000`
`continue`	`c`	继续运行到下一个断点
`stepi`	`si`	**执行 1 条机器指令**（汇编级单步）
`nexti`	`ni`	执行 1 条机器指令，但**跳过函数调用**
`step`	`s`	**执行 1 行 C 代码**（会进入被调函数）
`next`	`n`	执行 1 行 C 代码（不进入被调函数）
`info registers`	`i r`	看所有寄存器
`info registers <reg>`	`i r a0`	看指定寄存器
`x/<n><fmt><size> <addr>`	—	**看内存**。例：`x/8gx 0x80200000` 看 8 个 8 字节十六进制
`x/<n>i <addr>`	—	**看指令**。例：`x/5i $pc` 看当前位置 5 条指令
`print <表达式>`	`p`	求值。例：`p hartid`、`p/x $sp`、`p &kernel_end`
`backtrace`	`bt`	**看调用栈**（挂了的时候第一个该敲的命令）
`frame <n>`	`f`	切到第 n 层栈帧，看那一层的局部变量
`info frame`	—	看当前栈帧详情

命令	简写	作用
`disassemble <func>`	`disas`	反汇编某函数
`layout src`	—	分屏显示 C 源码（终端需支持）
`monitor info registers`	—	让 **QEMU** 报它的寄存器视图（有时比 gdb 的更真实）
`quit`	`q`	退出 gdb

🧠 **`x/` 命令的格式密码：** `x/` 后面接 `数量` + `格式` + `单位`：

```
x/8gx  0x80200000
  |||└─ x = 十六进制输出
  ||└── g = giant word（8 字节）
  |└─── 8 = 8 个
  └──── x = examine（检查内存）

常见单位：b=1字节  h=2字节  w=4字节  g=8字节
常见格式：x=十六进制  d=十进制  i=指令  s=字符串  c=字符
```

所以 `x/16bx $sp` = "从 sp 开始，看 16 个十六进制字节"，`x/4i $pc` = "从 pc 开始，看 4 条指令"。

⚠️ **`$sp` 里的 `$` 是什么？** 是 gdb 的**寄存器引用前缀**。`$pc`、`$sp`、`$a0` 都是"读这个寄存器的值"。这和 Makefile 的 `$`、shell 的 `$` 完全不是一回事——**三套 `$`，各管各的，别混。**

6.5 一个"实战排错"演练

⌨️ 试着手工制造一个 bug，用 gdb 抓出来：

① 在 `kernel/kalloc.c` 的 `kfree()` 里，把下界检查改错：

```
// 原本：if(((uint64)pa % PGSIZE) != 0 || (char*)pa < kernel_end || (uint64)pa >= PHYSTOP)
// 改成（故意改错，删掉下界检查）：
  if(((uint64)pa % PGSIZE) != 0 || (uint64)pa >= PHYSTOP)
    panic("kfree");
```

② `make run`，观察现象

> 大概率会 panic，或者更糟：**静默地把内核代码区当成空闲页分出去，然后在某个随机的时刻崩溃。**

③ 用 gdb 定位：

```
break panic
continue
# 停下后：
backtrace           ← 看谁调的 panic
frame 1
info locals         ← 看 pa 的值
p/x pa
# 拿到 pa 后，和 kernel_end 比一比：
p/x (char*)kernel_end
```

④ **想清楚**：为什么 `pa < kernel_end` 这个检查**绝对不能删**？

> 因为 `kfree` 的语义是"把一个**空闲物理页**还回分配器"。如果允许把**内核自己的代码页**还回去，那么：
> ① 内核代码被 `memset(pa, 1, PGSIZE)` **覆写成 0x01** → 内核自己被破坏
> ② 那个页会被 `kalloc` 分给某个进程 → 进程会读到/写到内核代码
> → **这是最严重的类型：内存破坏（memory corruption）。** 它会以"完全不相关的形式"在任何地方崩掉，极难定位。
> → 💡 **这就是内核里密集使用 `panic()` 的原因**：宁可当场明确地死，也不要带着损坏的状态苟活。

🧠 **顺便理解 `kfree` 里的 `memset(pa, 1, PGSIZE)` 和 `kalloc` 里的 `memset(r, 5, PGSIZE)`**：

- `kfree` 填 `1` = `0x01010101...` → 如果之后有人**用**了这个已释放的页，会读到 `0x0101...` 这种诡异值（而不是 0），**立刻暴露**
- `kalloc` 填 `5` = `0x05050505...` → 如果新分配的页被**误以为已初始化**就用了，也会读到诡异值

🧠 **这两个 `memset` 的学名是"毒化（poisoning）"**：故意写入**不可能合法出现的值**，让"用了未初始化/已释放的内存"这类 bug **更容易在测试时暴露**，而不是在生产环境里偶发。这是内核开发的经典手法。

---

7. 从源码到跑起来的完整链路（总图）

把前面所有内容串成一张图。**这张图是你 lab0 的最终检验标准：能照着它讲 5 分钟，lab0 就过关了。**

```
──────────────────────── 第 ① 层 源码层 ────────────────────────

  kernel/entry_qemu.S     第一条指令 _entry（汇编）
  linker/qemu.ld          规定：内核放 0x80200000，入口是 _entry
  kernel/*.c              main / vm / proc / kalloc / trap / syscall ...
  kernel/include/*.h      memlayout.h(KERNBASE/PHYSTOP)  riscv.h(PGSIZE/SATP)
  Makefile                CFLAGS / TOOLPREFIX / 三个目标
  xv6-user/*.c            用户程序：init / sh / ls / cat ...（+ lab1 的 halt）
  xv6-user/usys.pl        生成 syscall 用户态桩的脚本

──────────────────────── 第 ② 层 构建层 ────────────────────────

  make build
      │
      ├─► riscv64-unknown-elf-gcc 把 .c/.S 编成 .o（-mcmodel=medany -nostdlib ...）
      ├─► riscv64-unknown-elf-ld  按 qemu.ld 把 .o 链成 target/kernel
      ├─► objdump -S   → target/kernel.asm    （汇编/C 对照，读内核的武器）
      ├─► objdump -t   → target/kernel.sym    （符号表，查地址的字典）
      └─► perl usys.pl → usys.S → usys.o      （生成 syscall 桩）
          每个用户程序：xxx.c → xxx.o ─┐
                                       ulib.o      ├─► ld -Ttext 0 → _xxx
                                       usys.o      │
                                       printf.o    │
                                       umalloc.o   ─┘

  make fs
      │
      └─► dd 造空镜像 → mkfs.vfat -F 32 → sudo mount /mnt
          → 循环 cp _xxx → /mnt/xxx 和 /mnt/bin/xxx
```

```
          → sudo umount → fs.img 就绪（FAT32）

──────────────────────────── 第 ③ 层 运行层 ────────────────────────────

  make run  ⇒  qemu-system-riscv64 -machine virt -kernel target/kernel \
                   -m 128M -nographic -smp 2 -bios default \
                   -drive file=fs.img,... -device virtio-blk-device,...

      ┌──────────────────────────────────────────────────────────────┐
      | ① QEMU 加电    PC = 0x1000                                   |
      |      ↓                                                       |
      | ② OpenSBI 固件在 0x80000000 跑（M 模式），a0=hartid, a1=dtb   |
      |      ↓                                                       |
      | ③ 跳内核入口 _entry @ 0x80200000（★ gdb 里 break *_entry）    |
      |      mov t0 = hartid+1; t0 <<= 14; sp = boot_stack + t0       |
      |      call main                                               |
      |      ↓                                                       |
      | ④ main(hartid, dtb_pa):           （★ gdb 里 break main）    |
      |      consoleinit / printfinit / print_logo    ← 你改的横幅在这 |
      |      kinit()        ← 建空闲页链表；打印 kernel_end/phystop    |
      |      kvminit()      ← 建内核页表（★ lab1 专题 2）              |
      |      kvminithart()  ← w_satp() 开分页 ★ 地址从此变"虚拟"       |
      |      timerinit / trapinithart / procinit / plicinit...       |
      |      binit / fileinit / userinit   ← 造第一个进程（★ lab1 专题 3）|
      |      scheduler()    ← 再也不返回，进调度循环                   |
      |      ↓                                                       |
      | ⑤ 第一次进用户态：initcode 在虚拟地址 0，执行 ecall exec("/init")|
      |      ↓                                                       |
      | ⑥ init 起 shell，屏幕出现  -> / $                            |
      └──────────────────────────────────────────────────────────────┘
```

🎯 lab0 的最终检验：指着这张图，从 `make run` 讲到 `-> / $`，每一步说清"在哪个文件、哪个地址、哪个特权态"。

> 你会发现在 ④ 里的 `kvminit` / `userinit` 和 ⑤ 讲不透——**那正是 lab1 前六个专题的入口**。lab0 只要能把"这里有个洞"指出来，就完成了它的使命。

---

8. 思考题（不计分，但这才是 lab0 的真正内容）

> 建议先自己写答案再看括号里的提示。**答不上来是正常的，说明你找到了下一层该学的东西。**

🌱 基础（器层）

1. `TOOLPREFIX` 改成 `riscv64-linux-gnu-` 会怎样？为什么本课程必须用 `riscv64-unknown-elf-`？

   > 提示：`-nostdlib`、`-ffreestanding` 已经说明内核不链标准库。那换工具链会引入什么？

2. 为什么**编译器**的 warning 会让 `make build` 失败，而**链接器**的 `LOAD segment with RWX permissions` warning 天天出现、构建却照样成功？

   > 提示：`-Werror` 管得住 gcc，管得住 ld 吗？

3. `make clean` 为什么不删 `fs.img`？什么时候你应该手动删它？

🌿 **进阶（技层）**

4. 你在 `kernel/logo.c` 里改了横幅文字，`make run` 却毫无变化；改 `kernel/printf.c` 的 `print_logo()` 立刻生效。为什么？

   > 提示：去 `Makefile` 的 `OBJS` 里找 `logo.o`；再想想 `-D QEMU` 决定的是哪套 `print_logo` 被编进去。

5. `_entry` 的前两条指令为什么必须避免使用栈？
6. 把 `-smp 2` 改成 `-smp 4`，`entry_qemu.S` 的哪一行必须同步改？改成多少？

   > 提示：`.space 4096 * 4 * 2` 这个表达式的三个因子分别代表什么？

7. `kernel.asm` 里 `_trampoline` 的地址为什么必须是 `0x1000` 的整数倍？
8. `fs.img` 里的用户程序被放在两个位置（`/xxx` 和 `/bin/xxx`）。这是为什么？

   > 提示：xv6 的 `init` 怎么找 shell？shell 里敲 `ls` 时按什么顺序找可执行文件？

🌳 **拔高（术/法/道层，lab1 会展开）**

9. **`kvminithart()` 之后，`_entry` 里那句 `la sp, boot_stack` 再执行一次，拿到的是同一个值吗？**

   > 提示：`la` 生成的是"以 PC 为基准的相对寻址"还是"绝对地址"？开分页前后，同一个符号对应的虚拟地址和物理地址一样吗？
   > → 这是 lab1 专题 2 的核心。

10. **`kfree()` 的 `memset(pa, 1, PGSIZE)` 和 `kalloc()` 的 `memset(r, 5, PGSIZE)` 是"必要的功能"还是"调试辅助"？如果删掉，程序还能正确运行吗？**

    > 提示：区分"**正确性**"和"**可调试性**"。这两个 `memset` 属于哪一类？

11. **为什么 gdb 要设计成"前端/后端分离"的远程协议，而不是直接读内存？**

    > 提示：同一个 gdb 要同时能调本机进程、QEMU 虚拟机、真实开发板。如果把"读内存"写死在 gdb 里，会怎样？

12. **`ASSERT`（链接期检查）和 `panic()`（运行期检查）是同一个思路吗？**

    > 提示：它们都在做"尽早失败"。区别在于**能检查的时间窗口**。举一个"只能运行期检查"的例子（提示：用户输入的指针）。

---

9. 常见问题 FAQ

Q1 `make build` 报 `riscv64-unknown-elf-gcc: command not found`

不是代码问题，是环境问题。 依次排查：

```
which riscv64-unknown-elf-gcc          # 有没有?
ls /usr/bin | grep riscv               # 装了但叫别的名字?
echo $PATH                             # 装了但不在 PATH?
```

预装虚拟机里如果确实没有，找老师/助教确认虚拟机的工具链安装位置，可能需要 `export PATH=...:$PATH`。

Q2 `make fs` 报 `mount: /mnt: permission denied` 或要求 sudo 密码

`mount` 是特权操作。**在虚拟机的终端里**（不是在这个 AI 会话里）执行 `make fs`，输入密码即可。
🚫 **不要**试图用 `sudo` 包装整个 make（`sudo make run`）——会导致构建产物变成 `root` 所有，之后普通用户就改不动了，非常麻烦。

Q3 `make run` 黑屏，什么都不打印

按顺序查：

1. `fs.img` 存在吗？`ls -l fs.img`（不存在 → 先 `make fs`）
2. QEMU 报错了吗？（看 `make run` 输出里 QEMU 自己的报错行）
3. `-smp 2` 下两个 hart 的栈够吗？（应该是够的）
4. 你有没有改坏 `kernel/main.c` 的开头？
5. **QEMU 启动后按回车**——有时只是提示符没刷出来

Q4 改了代码但运行结果没变

三步走：

```
make -n build | grep <你改的文件>     # ① make 认不认为需要重编？
make clean && make run                # ② 强制全量重建
# ③ 改的是 xv6-user/ 下的文件吗？→ 必须重新 make fs
```

Q5 QEMU 退不出来

`Ctrl+A` **松开**，再按 `X`。
如果还不行：另开一个终端 `pkill -f qemu-system-riscv64`。

Q6 `qemu-gdb` 起了但 gdb 连不上 1234 端口

1. QEMU 那一侧真的加了 `-s` 吗？看 `make -n qemu-gdb` 展开的命令行
2. 端口被别的进程占了？`lsof -i :1234`
3. gdb 里的 `target remote localhost:1234` 写对了吗？

Q7 gdb 里 `break main` 报 `Function "main" not defined`

`target/kernel` 的调试信息没被读到。检查：

1. `make gdb` 里的 `file target/kernel` 路径对不对
2. `target/kernel` 是刚构建的吗（不是 `make clean` 之后又没构建）
3. `CFLAGS` 里的 `-g -ggdb` 还在吗（**不要删！**）

Q8 报 `Makefile:XX: *** missing separator. Stop.`

这一行的缩进用了空格，必须用 TAB。 在编辑器里打开"显示不可见字符"，把行首空格换成 TAB。

Q9 `objdump`/`objcopy` 的 `-S` 到底什么意思

看是哪个工具：

- `objcopy -S` = **strip**（去掉符号）
- `objdump -S` = **source intermix**（源码与汇编交替）

同一个字母，两个工具，两种含义。 记不住就问 `man`。

Q10 我能不能在 macOS / Windows 上做这个实验

不建议。 本课程统一用**预装好的虚拟机**，原因就是：

- 交叉工具链和 QEMU 版本敏感（`-machine virt`、OpenSBI 版本都会影响启动）
- `make fs` 需要 `mount` + `mkfs.vfat` + FAT32（Linux 上最顺）
- 统一环境 = 排错时老师能对照，你自己瞎折腾半天老师帮不上

如果你非要在别的系统上做：那就不是在学 lab0 的内容了，而是在搭环境——那不属于本实验的目标。

---

10. 验收标准

必做（全过才算通过 lab0）

#	验收项	怎么证明
1	工具链自检通过	贴出 `riscv64-unknown-elf-gcc --version` 和 `qemu-system-riscv64 --version` 的输出
2	`make build` 成功	无 `Error`/`error:`，且 `ls target/kernel` 存在
3	`make fs` 成功	`ls -lh fs.img` 有合理大小（约 256MB，因为 `bs=512k count=512`）
4	`make run` 看到 shell	截屏或贴出 `-> / $` 前后各 10 行输出
5	**动手 A** 自定义横幅生效	贴出你改后的 `kernel/printf.c` 中 `print_logo()` 片段 + 启动输出里的横幅
6	**动手 B** 内存计算	贴出你算的表：内核大小 / 管理区间 / 空闲页数，并指出 `kernel_end` 实测值
7	**动手 C** gdb 能停住	贴出：`break *_entry` → `continue` 后 `info registers` 的 `pc` 值，以及单步 5 条后 `sp` 的最终值
8	思考题 1~8 作答	手写/打字皆可，**不要求全对，要求写了自己的推理**

加分（选做）

#	加分项
+1	思考题 9~12 中的任意两题，写出有依据的推理
+2	在 gdb 里跟踪 `main()` 里的**全部**初始化调用，按顺序列出 13 个函数名
+3	用 `x/16gx` 查看 `fs.img` 在虚拟机内存里的内容（提示：需要一个 virtio 驱动读过的地址）
+4	让 `mode := release` 下的 `make run` 也能打印 `kernel_end`（提示：不要改 `#ifdef DEBUG`，而是改 Makefile）

明确不要求

- ❌ 不要求读懂 `vm.c` / `proc.c` / `trap.c` 的实现（**那是 lab1 的活**）
- ❌ 不要求会写 Makefile（**能读懂 + 会加一个 target 就够了**）
- ❌ 不要求懂 RISC-V 汇编（**能看懂 `_entry` 那 5 条就够**）
- ❌ 不要求重装工具链（**虚拟机里都有**）

> 🎯 **一句话验收：指着一张白纸，从 `make run` 讲到 `-> / $`，讲清每一步在哪个文件、落在哪个地址、用没用栈。讲得出来，lab0 就过。**

附录 A：命令速查表

构建与运行

```
make build              # 编译内核 + 全部用户程序
make fs                 # 生成/刷新 fs.img（需 sudo）
make run                # build + fs + 启动 QEMU
make clean              # 删构建产物（保留 fs.img）
make -n <target>        # 只打印将执行的命令，不执行（神器）
make -j4 build          # 4 路并行编译（快，但出错信息会交错）

# 手动删 fs.img（怀疑镜像坏了时）
rm fs.img
```

退出 QEMU

```
Ctrl + A ，松开，再按 X
```

三个宝：kernel.asm / kernel.sym / fs.img

```
# 查符号地址
grep -E " _entry$| main$| kernel_end$| etext$| _trampoline$" target/kernel.sym

# 看某个函数的汇编（-A N = 匹配行及其后 N 行）
grep -A 30 '<main>:' target/kernel.asm

# 看镜像挂了没
ls -lh fs.img
```

gdb 工作流

```
# 终端 A
make qemu-gdb

# 终端 B
make gdb
#  然后在 gdb 里:
#   break *_entry / continue / info registers
#   x/5i $pc / stepi / x/8gx 0x80200000
```

关键地址（背下来）

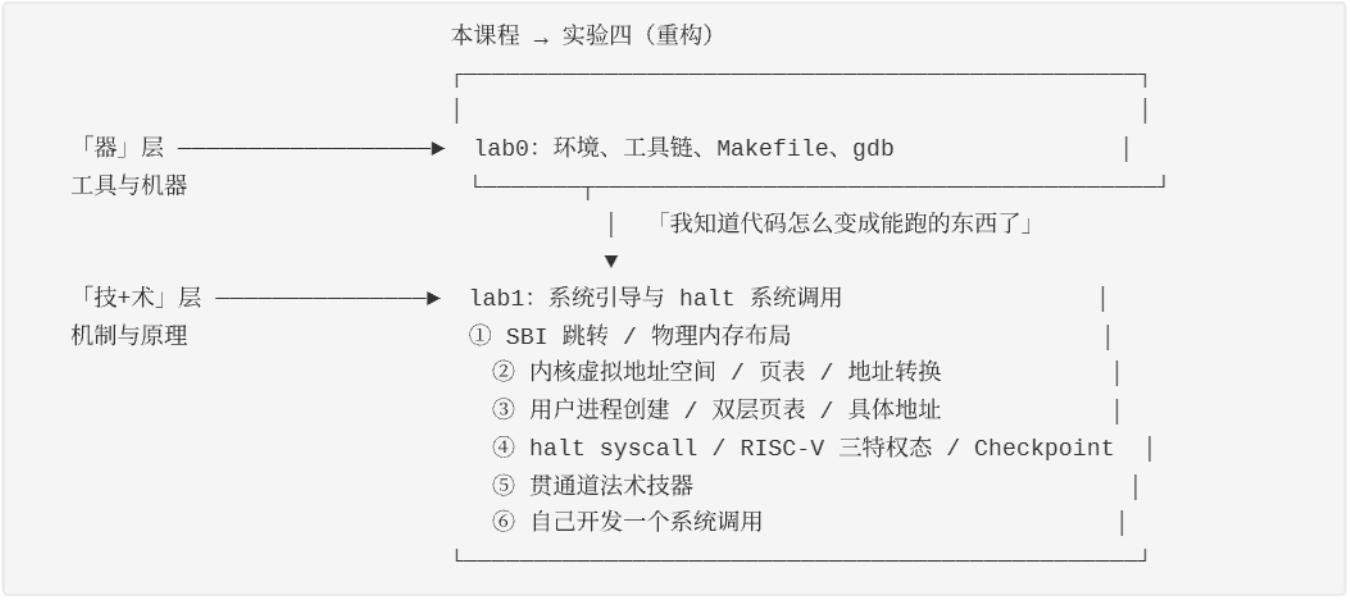
名称	值	含义
QEMU virt DRAM 起点	`0x80000000`	物理内存从这里开始
OpenSBI 固件	`0x80000000`	M 模式，占 2MB
KERNBASE	`0x80200000`	xv6 内核起点，也是 `_entry` 的地址
PHYSTOP	`0x80600000`	xv6 管理的物理内存上界
`-m 128M` 上界	`0x88000000`	QEMU 给的内存终点（xv6 只用前 6MB）
UART0	`0x10000000`	串口（你的屏幕输出）
VIRTIO0	`0x10001000`	磁盘（`fs.img`）
CLINT	`0x02000000`	定时器 / 核间中断
PLIC	`0x0c000000`	外部中断控制器
QEMU 复位向量	`0x1000`	CPU 加电后 PC 指这里

关键常量（`kernel/include/`）

```
// riscv.h
#define PGSIZE      4096          // 一页 4KB
#define MAXVA       (1L << 38)    // 虚拟地址空间顶点（lab1 细讲）
#define SATP_SV39   (8L << 60)    // 开启 Sv39 分页

// memlayout.h
#define VIRT_OFFSET 0x3F00000000L // 内核直映射的虚拟/物理偏移（lab1 细讲）
#define KERNBASE    0x80200000L   // 内核物理起点
#define PHYSTOP     0x80600000L   // 物理内存管理上界
#define TRAMPOLINE  (MAXVA - PGSIZE)   // trampoline 的虚拟地址（lab1 细讲）
```

附录 B：lab0 在整门课里的位置



lab0 与 lab1 的分工：

	lab0	lab1
关注点	**代码怎么变成能跑的东西**（器）	**跑起来之后发生了什么**（技/术）
核心产物	环境验证报告 + 自制横幅 + gdb 实测记录	4 个 Checkpoint 的实现 + 六大专题报告
地址观	只讲**物理地址**（`0x80200000`）	讲**物理 + 虚拟两层**（`VIRT_OFFSET`、页表）
特权态	只提一句"有 M/S/U 三种"	**完整讲三态及转换指令**
不碰的	页表、进程、trap（只指出"这里有个洞"）	——

> lab0 结束时，你应该能明确说出**三个"我知道这里有东西但还没学"的洞**：
>
> 1. `kvminit()` 建的那个内核页表里**到底装了什么**？
> 2. `userinit()` 造的那个进程，**它的地址空间长什么样**？

> 3. `ecall` 之后，CPU **具体经过哪些指令**才回到内核？
>
> **能说出这三个洞，lab0 就真正完成了。**
